

Funkcje okna



LAG

Funkcje **LAG()** i **LEAD()** to funkcje okienkowe w SQL, które umożliwiają pobieranie wartości z poprzedniego lub następnego wiersza w zestawie wyników, zgodnie z określonym porządkiem. Oto, do czego służą:

- **LAG():**

Umożliwia pobranie wartości z wiersza poprzedzającego bieżący wiersz. Przydatna jest np. do porównywania bieżącej wartości z wartością z poprzedniego okresu lub rekordu, co umożliwia obliczanie różnic czy zmian.

- **LEAD():**

Umożliwia pobranie wartości z wiersza następującego po bieżącym wierszu. Dzięki temu można sprawdzić, jaką wartość ma kolejny rekord względem bieżącego, co też jest użyteczne przy analizach porównawczych i prognozowaniu trendów.

LAG

	AZ last_name ▼	hire_date ▼	AZ poprzednik ▼
1	Garcia	2011-01-13	----
2	Brown	2012-06-07	Garcia
3	Higgins	2012-06-07	Brown
4	Gietz	2012-06-07	Higgins
5	Jacobs	2012-06-07	Gietz
6	Faviet	2012-08-16	Jacobs
7	Gruenberg	2012-08-17	Faviet
8	Li	2012-12-07	Gruenberg

```
SELECT
    last_name,
    hire_date,
    LAG(last_name, 1, '----') OVER (ORDER BY hire_date) AS
    poprzednik
FROM
    employees
```

LAG - partitional

	A-Z last_name	123 department_id	hire_date	A-Z lag
1	Whalen	10	2013-09-17	[NULL]
2	Martinez	20	2014-02-17	[NULL]
3	Davis	20	2015-08-17	Michael Martinez
4	Li	30	2012-12-07	[NULL]
5	Khoo	30	2013-05-18	Den Li
6	Tobias	30	2015-07-24	Alexander Khoo
7	Baida	30	2015-12-24	Sigal Tobias
8	Himuro	30	2016-11-15	Shelli Baida
9	Colmenares	30	2017-08-10	Guy Himuro
10	Jacobs	40	2012-06-07	[NULL]

SELECT

```
last_name,  
department_id,  
hire_date,  
LAG (CONCAT (first_name, ' ', last_name), 1)  
      OVER ( PARTITION by department_id  
              ORDER BY hire_date )
```

FROM

employees



LEAD

	A-Z last_name ▼	🕒 hire_date ▼	A-Z następny ▼
1	Garcia	2011-01-13	Brown
2	Brown	2012-06-07	Higgins
3	Higgins	2012-06-07	Gietz
4	Gietz	2012-06-07	Jacobs
5	Jacobs	2012-06-07	Faviet
6	Faviet	2012-08-16	Gruenberg
7	Gruenberg	2012-08-17	Li

```
SELECT
    last_name,
    hire_date,
    LEAD (last_name, 1, '----')
        OVER (ORDER BY hire_date) AS
        następny
FROM employees
```

LEAD – partitional

	A-Z last_name ▼	🕒 hire_date ▼	123 📄 department_id ▼	A-Z następny ▼
1	Whalen	2013-09-17	10	[NULL]
2	Martinez	2014-02-17	20	Davis
3	Davis	2015-08-17	20	[NULL]
4	Li	2012-12-07	30	Khoo
5	Khoo	2013-05-18	30	Tobias
6	Tobias	2015-07-24	30	Baida
7	Baida	2015-12-24	30	Himuro
8	Himuro	2016-11-15	30	Colmenares
9	Colmenares	2017-08-10	30	[NULL]

```
SELECT
    last_name,
    hire_date,
    department_id,
    LEAD(last_name, 1)
      OVER (partition by department_id
            ORDER BY hire_date) AS następny
FROM employees
```



STRING_AGG

Funkcja **STRING_AGG** to funkcja agregująca w SQL, która umożliwia łączenie (konsolidację) wielu wartości tekstowych z kolumny w jeden ciąg znaków. W trakcie agregacji między poszczególnymi wartościami można wstawić określony separator, co ułatwia prezentację danych w postaci listy lub zestawienia.

Kluczowe zastosowania i cechy:

- **Agregacja tekstu:** Umożliwia złączenie wielu wierszy w jeden ciąg tekstowy, co jest przydatne np. przy tworzeniu listy elementów, tagów czy nazw.
- **Określony separator:** Użytkownik może zdefiniować separator, który będzie wstawiany między poszczególnymi wartościami (np. przecinek, średnik, spacja).
- **Łatwość użycia:** Pozwala uniknąć konieczności stosowania bardziej skomplikowanych rozwiązań, takich jak pętle czy podzapytania do konsolidacji danych.

STRING_AGG

	123 department_id	A-Z nazwiska
1	20	Martinez,Davis
2	50	Weiss,Fripp,Kaufling,Vollman,Mourgos,Nayer,Mikkilineni,Landry,Markle,B
3	30	Li,Khoo,Baida,Tobias,Himuro,Colmenares
4	110	Higgins,Gietz
5	70	Brown
6	100	Gruenberg,Faviet,Chen,Sciarra,Urman,Popp
7	[NULL]	Grant
8	80	Singh,Partners,Errazuriz,Cambrault,Zlotkey,Tucker,Bernstein,Hall,Olsen,Ce
9	90	King,Yang,Garcia
10	60	James,Miller,Williams,Jackson,Nguyen
11	10	Whalen
12	40	Jacobs

```
SELECT
    department_id,
    STRING_AGG(last_name,',') nazwiska
from employees
group by department_id
```


STRING_AGG - sortowanie

	123 department_id	A-Z nazwiska
1	10	Whalen
2	20	Davis,Martinez
3	30	Baida,Colmenares,Himuro,Khoo,Li,Tobias
4	40	Jacobs
5	50	Atkinson,Bell,Bissot,Bull,Cabrio,Chung,Davies,Dellinger,Dilly,Everett,
6	60	Jackson,James,Miller,Nguyen,Williams
7	70	Brown
8	80	Abel,Ande,Banda,Bates,Bernstein,Bloom,Cambrault,Cambrault,Dora
9	90	Garcia,King,Yang
10	100	Chen,Faviet,Gruenberg,Popp,Sciarra,Urman
11	110	Gietz,Higgins
12	[NULL]	Grant

SELECT

department_id,

STRING_AGG(last_name,', ' **order by** last_name)

nazwiska

from employees

group by department_id

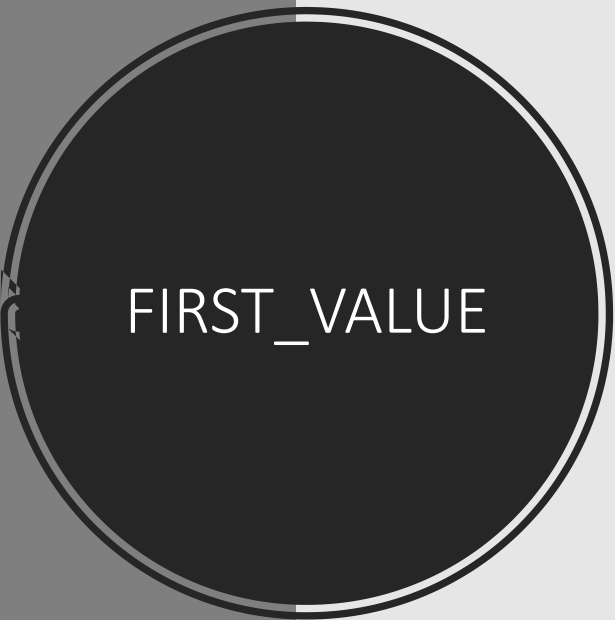
Odwrócone STRING_AGG - Split to table

	123 id ▼	A-Z tekst ▼
1	1	abc
2	1	opo
3	1	ddd
4	2	xxx
5	2	ccc

```
create table t1 (id int, tekst varchar(100))
```

```
insert into t1 values (1, 'abc,opo,ddd'), (2, 'xxx,ccc')
```

```
SELECT
    id,
    regexp_split_to_table(tekst, ',') AS tekst
FROM t1
```



FIRST_VALUE

Funkcja **FIRST_VALUE()** to funkcja okienkowa w SQL, która zwraca pierwszą wartość z określonego zbioru wierszy w ramach okna (partycji), według ustalonego porządku sortowania. Dzięki niej można uzyskać wartość odniesienia dla danej grupy, co jest przydatne m.in. przy:

- **Porównaniach wartości:** Możesz porównać bieżącą wartość z pierwszą wartością w danej grupie, aby np. obliczyć różnicę czy procentową zmianę.
- **Ustaleniu wartości bazowej:** Umożliwia wyodrębnienie "pierwszego" rekordu, który może służyć jako punkt odniesienia w analizach.
- **Raportowaniu:** Pozwala na łatwe przedstawienie najwcześniejszej lub minimalnej (przy odpowiednim sortowaniu) wartości w ramach każdej partycji.

FIRST_VALUE

	123 department_id	A-Z last_name
1	30	Li
2	70	Brown
3	20	Martinez
4	10	Whalen
5	80	Singh
6	60	James
7	100	Gruenberg
8	90	King
9	40	Jacobs
10	50	Fripp
11	110	Higgins
12	[NULL]	Grant

```
SELECT
    distinct department_id,
    FIRST_VALUE(last_name)
        over (partition by department_id
              order by salary DESC) last_name
from employees
```



LAST_VALUE

Funkcja **LAST_VALUE()** to funkcja okienkowa w SQL, która zwraca ostatnią wartość z określonego zbioru wierszy w obrębie zdefiniowanego okna (partycji), według ustalonego porządku sortowania.

Kluczowe aspekty:

- **Wyodrębnianie ostatniej wartości:**

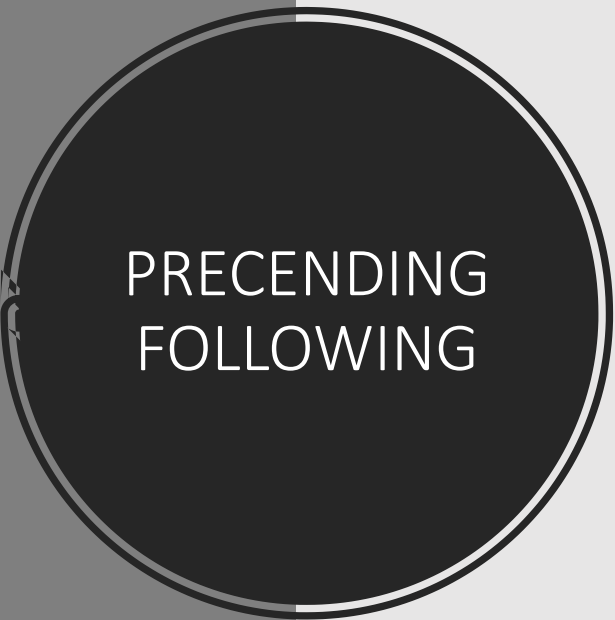
Funkcja ta pozwala pobrać wartość z ostatniego wiersza w danej partycji, co jest przydatne przy porównywaniu wyników lub wyciąganiu wartości końcowej z szeregu danych.

- **Definicja okna:**

Podobnie jak w przypadku innych funkcji okienkowych, do LAST_VALUE() używamy klauzuli OVER z opcjonalnym PARTITION BY oraz ORDER BY.

Należy jednak pamiętać, że domyślnie okno może być zdefiniowane jako "od początku partycji do bieżącego wiersza". Aby uzyskać wartość ostatniego wiersza całej partycji, warto jawnie określić zakres okna, np.:

ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING



PRECEDING FOLLOWING

Operatory **PRECEDING** i **FOLLOWING** są używane w klauzuli określającej ramy okna (ang. window frame) w funkcjach okienkowych SQL. Umożliwiają one zdefiniowanie zakresu wierszy, na których mają zostać wykonane obliczenia względem bieżącego wiersza, zgodnie z kolejnością określoną w klauzuli ORDER BY.

Kluczowe informacje:

- **PRECEDING:**
 - Odnosi się do wierszy, które występują **przed** bieżącym wierszem.
 - Można określić liczbę wierszy, np. *3 PRECEDING* oznacza trzy wiersze poprzedzające bieżący.
 - Specjalna wartość *UNBOUNDED PRECEDING* oznacza wszystkie wiersze od początku partycji aż do bieżącego wiersza.
- **FOLLOWING:**
 - Odnosi się do wierszy, które występują po bieżącym wierszu.
 - Można określić liczbę wierszy, np. *2 FOLLOWING* oznacza dwa wiersze następujące po bieżącym.
 - Specjalna wartość *UNBOUNDED FOLLOWING* oznacza wszystkie wiersze od bieżącego wiersza do końca partycji.

LAST_VALUE

	123 department_id	AZ last_name
1	10	Whalen
2	20	Davis
3	30	Colmenares
4	40	Jacobs
5	50	Olson
6	60	Nguyen
7	70	Brown
8	80	Kumar
9	90	Garcia
10	100	Popp
11	110	Gietz
12	[NULL]	Grant

```
SELECT
    distinct department_id,
    LAST_VALUE(last_name) over (partition by department_id
                                order by salary DESC RANGE BETWEEN
                                UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) last_name
from employees
order by department_id
```

WINDOW declaration

	123 department_id	123 suma
1	80	304,500
2	60	28,800
3	110	20,308
4	[NULL]	7,000
5	50	156,400
6	90	58,000
7	100	51,608
8	70	10,000
9	20	19,000
10	40	6,500
11	10	4,400
12	30	24,900

```
select
    distinct department_id,
    sum(salary) over (partition by department_id) suma
from employees
```


WINDOW declaration

	hire_date	123 salary	123 grow
1	2011-01-13	17,000	27,000
2	2012-06-07	10,000	39,008
3	2012-06-07	12,008	30,308
4	2012-06-07	8,300	26,808
5	2012-06-07	6,500	23,800
6	2012-08-16	9,000	27,508
7	2012-08-17	12,008	32,008
8	2012-12-07	11,000	30,908
9	2013-05-01	7,900	22,000

```
select
    hire_date,
    salary,
    sum(salary) over (order by hire_date
                      ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) grow
from employees
```

WINDOW declaration

	hire_date	salary	suma_krocza
1	2011-01-13	17,000	17,000
2	2012-06-07	10,000	27,000
3	2012-06-07	12,008	39,008
4	2012-06-07	8,300	47,308
5	2012-06-07	6,500	53,808
6	2012-08-16	9,000	62,808

```
select
  hire_date,
  salary,
  sum(salary) over (order by hire_date
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
    suma_krocza
from employees
```

WINDOW declaration

	hire_date	salary	grow
1	2011-01-13	17,000	[NULL]
2	2012-06-07	10,000	17,000
3	2012-06-07	12,008	27,000
4	2012-06-07	8,300	22,008
5	2012-06-07	6,500	20,308
6	2012-08-16	9,000	14,800
7	2012-08-17	12,008	15,500
8	2012-12-07	11,000	21,008

```
select
    hire_date,
    salary,
    sum(salary) over (order by hire_date
                      ROWS BETWEEN 2 PRECEDING AND 1 PRECEDING) grow
from employees
```

WINDOW declaration

	123 department_id	123 sum	123 okno
1	10	4,400	4,400
2	20	19,000	23,400
3	30	24,900	48,300
4	40	6,500	54,800
5	50	156,400	211,200
6	60	28,800	240,000
7	70	10,000	250,000
8	80	304,500	554,500
9	90	58,000	612,500
10	100	51,608	664,108
11	110	20,308	684,416
12	[NULL]	7,000	691,416

```
select
    department_id,
    sum(sum(salary)) over (order by department_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) okno
from employees
group by
    department_id
```